

Technical PRD: "Boardroom" (MVP)

1. Executive Summary

Vision: "Boardroom" is a web-based collaborative workspace that replicates the tactile freedom of a physical corkboard. Unlike rigid project management tools (like Trello/Jira), Boardroom allows free-form placement of visual assets. **Core Problem:** Digital organization tools often force linear structures (lists/grids). Users lose the spatial context and creative freedom of a physical board. **Value Proposition:** A persistent, free-form digital canvas where users can manipulate content directly, backed by a rigorous audit trail (History Log) to track every change.

2. User Personas & Stories

Primary Persona: The Organizer (Alex)

Student or Project Lead needing to visualize unstructured ideas.

Role	Action	Benefit
Organizer	Create a new Board	To start a dedicated space for a specific project.
Organizer	Drag and drop a sticky note	To arrange ideas spatially rather than linearly.
Organizer	View the "History Log"	To see who moved or deleted a specific note (accountability).

Secondary Persona: The Collaborator (Sam)

Team member contributing content.

Role	Action	Benefit
Collaborator	Log in via Google	Quick access without remembering new passwords.
Collaborator	Edit text on a note	To update information directly on the board interface.

3. Functional Requirements & Acceptance Criteria

P0: Critical MVP Features

Feature	Description	Technical Acceptance Criteria
Google Authentication	Secure login/signup using Google.	1. User clicks "Sign in with Google". 2. Account created in Firebase Auth. 3. Session persists on refresh.
The Board (Canvas)	A workspace to place items.	1. Canvas spans 100% viewport width/height. 2. Background replicates a texture (cork/grid).
Create Item (CRUD)	Add Notes/Images to board.	1. "Add Note" button spawns a div at center or clicked location. 2. Item persists in DB immediately on creation.
Direct Manipulation	Drag, Move, Resize items.	1. Items can be dragged to any (x,y) coordinate. 2. New coordinates save to DB <code>onDragEnd</code> (to reduce write volume).
In-Place Editing	Edit content directly.	1. Double-click item turns text into <code><input></code> or <code>textarea</code> . 2. Clicking outside (blur) saves changes.
History Log	Sidebar tracking all changes.	1. Every CRUD action creates a log entry. 2. Log displays: <code>Timestamp</code> , <code>User</code> , <code>Action</code> (e.g., "Moved Note A").

4. Implementation Roadmap

Phase 1: MVP Core (Weeks 1-4)

- **Week 1:** Setup React repo, Tailwind, and Firebase project. Implement Google Auth.

- **Week 2:** Build the "Canvas" and basic "Sticky Note" component using a drag-and-drop library.
- **Week 3:** Connect Canvas to Firestore (Database). Implement Save/Load of coordinates.
- **Week 4:** Implement the "History Log" logic and UI. Deploy.

Phase 2: Scaling (Future)

- Real-time collaboration (users see cursors moving live).
- Rich media support (PDF embedding, YouTube links).
- Multiple Boards per user.

5. Technical Architecture & UI/UX

For a **semi-beginner**, we will use a **Serverless / Backend-as-a-Service (BaaS)** architecture. This eliminates the need to write complex server code, allowing focus on the frontend logic.

Technology Stack

- **Frontend: React.js** (created via Vite). It is the industry standard.
- **Language: JavaScript** (easier for beginners) or **TypeScript** (better for long-term stability). *Recommendation: JavaScript for speed.*
- **Styling: Tailwind CSS.** Utility-first, allowing rapid styling without writing separate CSS files.
- **Backend & Database: Google Firebase.**
 - *Authentication:* Firebase Auth (Google Provider).
 - *Database:* Cloud Firestore (NoSQL, perfect for JSON-like board data).
 - *Hosting:* Firebase Hosting.
- **Drag & Drop Library:** `dnd-kit` or `react-draggable`. (Both are lightweight and React-friendly).

UI/UX Framework

- **Design System: Chakra UI** or **DaisyUI** (built on Tailwind). These provide pre-made components (Modals, Buttons, Sidebars) so you don't build them from scratch.
- **Layout:**
 - *Main View:* Full-screen canvas.
 - *Toolbar:* Floating bottom or side bar (Add Note, Add Image).
 - *History:* Collapsible drawer on the right side.

API Standard

Since we are using Firebase, we interact directly with the SDK (Software Development Kit) rather than a traditional REST API.

- **Pattern:** Event-driven. The app subscribes to the Firestore data; when data changes, the board updates automatically.

Security

- **Auth:** OAuth 2.0 via Firebase (Google Sign-In).
- **Rules:** Firestore Security Rules must be configured:

```
// Example Concept
allow read, write: if request.auth != null;
```

Infrastructure / DevOps

- **Hosting:** Firebase Hosting (Free tier is generous).
- **CI/CD:** Github Actions (optional) or manual deploy via Firebase CLI (`firebase deploy`).
- **Version Control:** Git.

6. Data Schema

Since we are using a NoSQL database (Firestore), the data is structured as Documents and Collections.

```
erDiagram
    USERS ||--o{ BOARDS : owns
    BOARDS ||--o{ ITEMS : contains
    BOARDS ||--o{ HISTORY_LOGS : has

    USERS {
        string uid PK "Firebase Auth ID"
        string email
        string displayName
        string photoURL
    }

    BOARDS {
        string id PK
        string ownerId FK
        string title
        timestamp createdAt
    }

    ITEMS {
        string id PK
```

```

        string boardId FK
        string type "text | image"
        string content "Text body or Image URL"
        int positionX
        int positionY
        string color
        timestamp lastModified
    }

    HISTORY_LOGS {
        string id PK
        string boardId FK
        string userId FK
        string actionType "CREATE | MOVE | EDIT | DELETE"
        string itemSummary "Note ID or preview text"
        timestamp timestamp
    }

```

7. User Flow & State Logic

```

flowchart TD
    A[Start] --> B{User Logged In?}
    B -- No --> C[Login Page / Google Auth]
    C --> B
    B -- Yes --> D[Dashboard / Board List]
    D --> E[Select Board]
    E --> F[Main Canvas View]

    subgraph Board_Actions
        F --> G[Action: Add Item]
        F --> H[Action: Drag Item]
        F --> I[Action: Edit Content]
    end

    G --> J[Update DB: Create Item]
    H --> K[Update DB: Update Coords]
    I --> L[Update DB: Update Content]

    J & K & L --> M[Trigger: Create History Log Entry]
    M --> N[Sync to UI (Real-time)]
    N --> F

```

8. Success Metrics (KPIs)

- **Latency:** Time from "Drop Item" to "Saved in DB" < 200ms.
- **Error Rate:** Failed save operations < 1%.